BÁO CÁO KỸ THUẬT

ASSIGNMENT 01 – DEVICE DRIVER

ioctl interface và misc counter device

Thông tin	Nội dung
Họ và tên	[Điền họ tên]
MSSV / Lớp	[Điền MSSV và lớp]
Môn học	Embedded Linux Systems
Nền tảng	Ubuntu – Linux 5.15 – ARMv7/QEMU vexpress-a9
Repository	Embedded-Linux-System / ASSIGNMENT01
Ngày hoàn thiện	[Điền ngày]

Tóm tắt

Bài tập gồm hai kernel module. Module asgn1_driver mở rộng character driver của LAB02 bằng bốn lệnh ioctl; counter_driver dùng miscdevice để tạo bộ đếm có thể đọc, ghi và xem thống kê qua procfs. Bài báo cáo tập trung vào cấu trúc chương trình, cách bảo vệ dữ liệu dùng chung và các trường hợp kiểm thử chính.

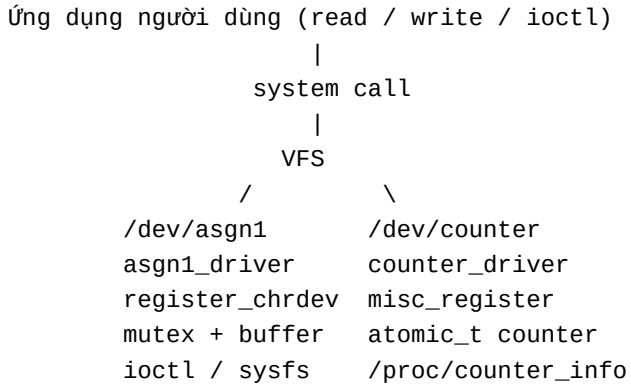
Bản ngắn gọn theo đề ASSIGNMENT 01. Cần điền thông tin cá nhân, bổ sung log gốc và tự vẽ lại sơ đồ kiến trúc trước khi nộp chính thức.

1. Môi trường và kiến trúc

1.1. Môi trường thực hiện

Mã nguồn được viết trên Ubuntu. Linux kernel 5.15 và các module được biên dịch chéo bằng arm-linux-gnueabihf-, sau đó chép vào initramfs để chạy trên QEMU vexpress-a9. LAB01 cung cấp kernel và rootfs; LAB02 là driver gốc dùng để mở rộng Task 1.

1.2. Luồng thao tác



Sơ đồ phác thảo: cần vẽ lại thành một hình kiến trúc riêng, có mũi tên ioctl → unlocked_ioctl → copy_to_user/copy_from_user, theo yêu cầu của đề.

Ở Task 1, VFS chuyển ioctl đến hàm unlocked_ioctl trong file_operations. Các lệnh GET_STATS và GET_VERSION dùng copy_to_user() để trả dữ liệu; SET_MODE dùng copy_from_user() để nhận tham số. Ở Task 2, misc framework tự quản lý major 10 và một minor; /proc/counter_info chỉ dùng để quan sát trạng thái.

1.3. Hai driver độc lập

asgn1_driver giữ một buffer ký tự 1024 byte và các bộ đếm thao tác. counter_driver quản lý giá trị số nguyên bằng atomic_t. Cả hai có cùng giao diện open/read/write, nhưng cách xử lý dữ liệu và đăng ký thiết bị khác nhau.

2. So sánh ba loại driver

Tiêu chí	Character	Block	Network
Giao diện	file_operations	Block layer / blk-mq	net_device_ops
Đơn vị dữ liệu	Byte stream	Sector / block	Packet / frame
Bộ đệm	Trong driver, tùy thiết kế	Page cache, hàng đợi I/O	SKB, hàng đợi TX/RX
Thiết bị ví dụ	UART, cảm biến, asgn1	SD, eMMC, SSD	Ethernet, Wi-Fi
Ứng dụng	Điều khiển, đọc ghi nhỏ	Lưu trữ có địa chỉ khối	Giao tiếp qua mạng

2.1. Vì sao chọn character/misc device?

Đối với /dev/asgn1, dữ liệu là chuỗi ngắn và các lệnh ioctl điều khiển trực tiếp trạng thái của driver. Vì vậy character device đơn giản và đủ dùng. /dev/counter cũng đọc ghi theo giao diện file nhưng chỉ có một nhiệm vụ; miscdevice giúp đăng ký nhanh hơn và không cần giữ một major riêng.

2.2. Hai trường hợp nên chọn subsystem khác

Nếu lưu nhật ký hệ thống lên eMMC hoặc thẻ SD, nên dùng block device cùng filesystem để tận dụng các cơ chế lưu trữ chuẩn. Nếu giao tiếp Ethernet, phải dùng network device vì dữ liệu đi theo frame và được xử lý qua network stack, không phải read/write một chuỗi tự do.

Việc chọn loại driver không phụ thuộc vào driver nào “mạnh” hơn mà phụ thuộc vào mô hình dữ liệu mà thiết bị cung cấp. Chọn đúng subsystem sẽ dễ bảo trì và giảm phần code phải tự viết.

3. Task 1 – ioctl character driver

Task 1 tiếp tục dùng register_chrdev() từ LAB02 với node /dev/asgn1 (major 241 trong cấu hình thử). Cấu trúc file_operations có open, release, read, write và unlocked_ioctl. Mutex bảo vệ buffer và các biến thống kê vì có thể có nhiều tiến trình truy cập đồng thời.

Lệnh	Macro	Xử lý chính
RESET_BUFFER	_IO	Xóa buffer, đặt buffer_len và read/write count về 0.
GET_STATS	_IOR	Đưa 5 trường thống kê về userspace bằng copy_to_user.
SET_MODE	_IOW	Nhận int 0/1, mode 0 thêm [KERNEL], mode 1 giữ nguyên.
GET_VERSION	_IOR	Trả chuỗi version trong vùng 64 byte.

3.1. Đồng bộ và xử lý lỗi

Mỗi lần GET_STATS, driver khóa mutex, chép giá trị sang struct tạm rồi mở khóa trước khi copy_to_user(). Với SET_MODE, mode phải được kiểm tra là 0 hoặc 1 trước khi lưu. Driver trả -EINVAL cho tham số sai, -ENOTTY cho lệnh ioctl không hỗ trợ và -EFAULT khi trao đổi dữ liệu với userspace thất bại.

3.2. Magic number và kích thước ioctl

```
#define ASGN1_IOC_MAGIC 'A'
#define ASGN1_RESET_BUFFER _IO(ASGN1_IOC_MAGIC, 0)
#define ASGN1_GET_STATS    _IOR(ASGN1_IOC_MAGIC, 1, struct asgn1_stats)
#define ASGN1_SET_MODE    _IOW(ASGN1_IOC_MAGIC, 2, int)
```

Magic number phân biệt nhóm lệnh. _IOC_NR() lấy số thứ tự trong mã ioctl; _IOC_SIZE() lấy kích thước dữ liệu mà macro đã mã hóa. Ví dụ GET_STATS có số thứ tự 1, còn kích thước bằng sizeof(struct asgn1_stats), dự kiến 20 byte nếu mỗi int là 4 byte.

4. Task 2 – misc counter driver

Counter driver đăng ký bằng misc_register(), chọn MISC_DYNAMIC_MINOR và tạo /dev/counter với major 10. Khi gỡ module, misc_deregister() thu hồi đăng ký. Driver còn tạo /proc/counter_info bằng proc_create() và xuất nội dung qua seq_file.

4.1. Đọc và ghi

Khi mở thiết bị, driver cấp phát trạng thái riêng cho file và lưu trong file->private_data. Lần đọc đầu trả giá trị hiện tại dưới dạng chuỗi có ký tự xuống dòng và tăng counter lên một đơn vị. Các lần read tiếp theo trên cùng file descriptor dùng *ppos để biết vị trí đã đọc; khi hết chuỗi, hàm trả 0 để cat dừng.

Khi ghi, driver nhận chuỗi ASCII, chuyển thành số bằng kstrtoint(), từ chối số âm hoặc chuỗi không hợp lệ và cập nhật counter bằng atomic_set(). Các thao tác ghi thành công được cộng vào total_writes.

4.2. Vì sao dùng atomic_t?

Nếu hai tiến trình cùng đọc và tăng một biến int bằng cách cộng thông thường, có thể xảy ra việc cả hai cùng nhìn thấy giá trị cũ, dẫn đến mất một lần tăng. atomic_fetch_inc() lấy giá trị cũ và tăng counter trong cùng một thao tác nguyên tử. Dù vậy, nhiều lệnh atomic đặt liền nhau không tự động thành một giao dịch chung.

4.3. Procfs và thu hồi tài nguyên

```
cat /dev/counter         # đọc một số và tăng counter
echo 100 > /dev/counter  # đặt counter bằng 100
cat /proc/counter_info   # xem giá trị và số lần đọc/ghi
```

Khi unload, module phải xóa proc entry rồi hủy đăng ký miscdevice. Nếu proc_create() thất bại sau khi misc_register() thành công, phần init phải gọi misc_deregister() trước khi trả lỗi. Bộ nhớ cấp trong open() cần giải phóng bằng kfree() ở release().

5. Kiểm thử trên QEMU

5.1. Task 1

Người thực hiện đã xác nhận bộ test tổng hợp Task 1 chạy PASS. Bảng dưới ghi các tiêu chí được kiểm tra; log gốc cần được lưu riêng trong ASSIGNMENT01/logs/test_asgn1_output.txt và dmesg_full.txt.

Kiểm thử	Kết quả cần đạt	Ghi nhận
RESET_BUFFER	Buffer rỗng, read/write count = 0	PASS (theo xác nhận)
GET_STATS	Năm trường thống kê đúng	PASS (theo xác nhận)
SET_MODE 0/1	[KERNEL]HELLO / HELLO	PASS (theo xác nhận)
Mode không hợp lệ	-EINVAL	PASS (theo xác nhận)
GET_VERSION	Nhận version hợp lệ, tối đa 64 byte	PASS (theo xác nhận)

5.2. Task 2

Các mốc nên lưu trong test_counter_output.txt: đọc liên tiếp hai lần; ghi 100 rồi đọc; ghi 0 rồi đọc; ghi chuỗi sai và số âm; xem /proc/counter_info trước/sau thao tác; nạp và gỡ module. Không chèn số liệu ví dụ thay cho output thật.

Mốc procfs cần chụp	Đường dẫn log
Trước ghi	[Dán output từ QEMU]
Sau ghi 100	[Dán output từ QEMU]
Sau ghi 0	[Dán output từ QEMU]

5.3. Lỗi đã gặp và cách xử lý

Trong lúc chạy test_version, open() từng báo “No such file or directory”. Cách kiểm tra là dùng lsmod, /proc/devices và ls -l /dev/asgn1. Khi module đã đăng ký major 241 nhưng node còn thiếu, có thể tạo node /dev/asgn1 đúng major/minor rồi thử lại. Lỗi nằm ở bước tạo node, không nhất thiết do ioctl.

6. Câu hỏi lý thuyết – phần 1

Câu 1. Reference counting của kernel module

Reference count cho biết module đang có những tham chiếu sử dụng. Khi file_operations.owner được đặt bằng THIS_MODULE, thao tác mở file thông thường giữ một tham chiếu; khi file descriptor đóng và tham chiếu được thả, số đếm giảm. Module cũng có thể tự tăng/giảm tham chiếu bằng các API phù hợp như try_module_get() và module_put().

Nếu một tiến trình vẫn giữ device đang mở, rmmod thường báo module đang bận. Cách xử lý an toàn là đóng file hoặc dừng tiến trình đang giữ file descriptor rồi thử unload lại. Một số kernel hỗ trợ force unload khi được cấu hình, nhưng biện pháp đó có nguy cơ gây lỗi hệ thống và không nên dùng trong bài thực hành.

Câu 2. Vì sao phải dùng copy_to_user()?

Một con trỏ đến từ userspace là địa chỉ ảo của tiến trình, không phải vùng nhớ kernel đã được xác thực. Vùng địa chỉ có thể chưa được ánh xạ, không đủ quyền ghi hoặc không chứa đủ số byte mong muốn. Ghi trực tiếp qua con trỏ đó bằng memcpy() có thể tạo page fault hoặc truy cập không an toàn ở chế độ kernel.

copy_to_user() và copy_from_user() cung cấp cơ chế truy cập bộ nhớ userspace phù hợp của Linux. Hàm có thể xử lý trường hợp truy cập lỗi và trả số byte chưa sao chép; driver đổi tình huống này thành -EFAULT. Vì vậy GET_STATS phải dùng copy_to_user() để chuyển struct thống kê ra chương trình gọi ioctl().

Điểm rút ra

Hai câu hỏi cùng nhấn mạnh ranh giới giữa phần mềm trong kernel và tiến trình userspace: tài nguyên module phải được giữ đúng vòng đời, còn dữ liệu đi qua ranh giới bộ nhớ phải được kiểm tra.

7. Câu hỏi lý thuyết – phần 2

Câu 3. MTD khác block device thế nào?

MTD quản lý flash thô như NAND/NOR, cho phép xử lý đặc điểm riêng của flash: erase block, bad block và ECC. Block device cung cấp giao diện truy cập theo sector, phù hợp với thiết bị như thẻ SD và eMMC, nơi bộ điều khiển lưu trữ đã che giấu phần lớn chi tiết flash nội bộ.

Ví dụ, một chip raw NAND nối trực tiếp vào SoC thường đi qua MTD và có thể dùng UBI/UBIFS. Còn eMMC thường xuất hiện dưới dạng /dev/mmcblk* ở block layer. Chọn MTD khi phần mềm phải trực tiếp quản lý đặc tính xóa/ghi của flash thô.

Câu 4. Vì sao read() phải xử lý *ppos?

Lệnh cat gọi read() lặp lại cho đến khi nhận 0 byte (EOF) hoặc lỗi. Nếu driver luôn trả cùng chuỗi và luôn báo số byte dương, cat sẽ in mãi. Với counter_driver, lần đọc đầu cần chuẩn bị một giá trị, trả dữ liệu và cập nhật offset. Khi *ppos đã tới cuối chuỗi, các lần read() tiếp theo trên cùng file descriptor phải trả 0.

Khi đóng rồi mở lại /dev/counter, driver có trạng thái file mới và lần đọc tiếp theo mới lấy giá trị counter kế tiếp. simple_read_from_buffer() là một cách xử lý offset và điều kiện EOF rõ ràng.

8. Kết luận

Qua hai module, bài tập cho thấy hai cách đăng ký character device và cách kết hợp file_operations với ioctl, atomic counter và procfs. Điểm cần chú ý nhất khi viết driver là bảo vệ trạng thái dùng chung, kiểm soát quyền truy cập bộ nhớ userspace và dọn tài nguyên khi có lỗi.

Tài liệu tham khảo

ASSIGNMENT-01 – How to Write a Device Driver; Linux Kernel 5.15 source; Linux Kernel Documentation (misc devices, atomic_t, procfs).

Trước khi nộp: điền MSSV; thay sơ đồ phác thảo bằng sơ đồ tự vẽ; đính kèm log QEMU thật; kiểm tra lại với bản code cuối cùng.